

SIGAH — Sistema Integrado de Gestão de Apoio Habitacional

Arquitetura Frontend · v3

*Documento de referência para engenheiro / IA responsável pela implementação.
Cobre exclusivamente a disposição de pastas e organização do projeto frontend.
Regras de negócio — classificação, cotas, prazos processuais, integrações externas — são responsabilidade exclusiva do backend e não aparecem neste documento.*

Fronteira frontend / backend

O frontend consome dados já processados pelo backend via API. Nunca calcula, nunca acessa sistemas externos, nunca toma decisões de negócio.

Stack tecnológico

Camada	Tecnologia	Papel
Framework	Next.js 14 (App Router)	Roteamento, SSR/SSG, layouts aninhados
Estilo	Tailwind CSS	Utilitários de estilo, tema via CSS vars
Componentes	shadcn/ui	Primitivos acessíveis, base para extensões de domínio
Linguagem	TypeScript	Tipagem estrita em toda a base de código

Princípio central de flexibilidade

O sistema atende múltiplos programas habitacionais. A flexibilidade no frontend é resolvida em dois níveis, ambos restritos a decisões de interface:

Nível	Quando usar	Mecanismo
1 — UI Config	Rótulos, tema visual ou seções da interface	Configuração por programa + ProgramContext
2 — Feature flag de UI	Uma seção ou componente aparece ou não	ProgramUIConfig lidas via useProgram()

Nível	Quando usar	Mecanismo
3 — Formulário modular	O atributo central do cadastro tem estrutura diferente	Configuração por programa + ProgramContext

Estrutura de pastas

```
sigh-frontend/
├─ app/
│   └─ [programId]/
│       ├── layout.tsx
│       ├── dashboard/page.tsx
│       ├── cadastro/
│       │   ├── page.tsx
│       │   └─ [familiaId]/page.tsx
│       ├── classificacao/page.tsx
│       ├── sorteio/page.tsx
│       ├── suplencia/page.tsx
│       ├── convocacao/page.tsx
│       ├── [candidatoId]/page.tsx
│       ├── empreendimentos/
│       ├── unidades/
│       ├── auditoria/logs/page.tsx
│       └─ relatorios/page.tsx
│   └─ auth/
│       └─ page.tsx
├─ components/
│   └─ forms/
│       ├── FamilyForm.tsx
│       └─ PropertyForm.tsx
│   └─ display/
│       ├── ScoringPanel.tsx
│       ├── QuotaDisplay.tsx
│       ├── DeadlineStatus.tsx
│       └─ IntegrationStatus.tsx
│   └─ timeline/ (ProcessTimeline, AuditLog)
│   └─ layout/ (FeatureGate.tsx)
│   └─ ui/ (DocumentUpload, ConsentBanner)
├─ contexts/ (ProgramContext, UserContext)
├─ lib/
│   ├── api/ (client.ts, programs.ts, families...)
│   ├── validations/
│   ├── formatters/
│   └─ hooks/ (useProgram, usePermissions)
└─ types/ (program.ts, family.ts, api.ts)
```



Camadas e suas responsabilidades

- contexts/ProgramContext** — config de UI do programa ativo. `ProgramUIConfig` é um objeto leve retornado pelo backend. Não contém regras de negócio, percentuais de cota, critérios de pontuação nem prazos processuais.
- app/[programId]/** — roteamento dinâmico por programa. O `layout.tsx` busca a config, instancia o Provider e protege as rotas autenticadas.
- components/forms/** — único módulo com variação estrutural. `FamilyForm.tsx` e `PropertyForm.tsx` controlados por feature flags.
- components/display/** — exibição encapsulada de dados da API. Ex: `ScoringPanel` recebe pontuação já calculada pelo backend.

Modularização — components/forms/

- No frontend deste projeto, há um único caso legítimo de modularização: o formulário de cadastro. Os demais (scoring, integrations, quotas, deadlines) foram corretamente realocados ao backend.
- `FamilyForm.tsx` → cadastro orientado à família.
 - `PropertyForm.tsx` → cadastro orientado ao imóvel.

Mapa de responsabilidade por requisito

RF	Nome resumido	Frontend faz	Backend faz
001	Cadastrar Família	<code>FamilyForm</code> com seções condicionais	Valida elegibilidade, persiste
002	Integrar CadÚnico	Exibe status retornado pela API	Consulta e valida CadÚnico
008/055	Classificar / Hierarquizar	<code>ScoringPanel</code> exibe pontuação	Executa algoritmo de classificação
011/012	Cotas de vagas	<code>QuotaDisplay</code> exibe distribuição	Calcula e aplica cotas por portaria
017	Prazos Processuais	<code>DeadlineStatus</code> exibe indicador	Monitora, calcula e enforce prazos

RF	Nome resumido	Frontend faz	Backend faz
040	LGPD / Consentimento	ConsentBanner coleta aceite	Registra consentimento com timestamp
076	Governança / Perfis	FeatureGate oculta/exibe por perfil	Valida permissões em cada operação

Verde = responsabilidade do frontend

Âmbar = responsabilidade do backend

Pontos de atenção e riscos de implementação

⚠ 1 — Latência da ProgramUIConfig (gargalo de renderização)

O layout.tsx de [programId] espera a config do backend. Usar cache stale-while-revalidate no Next.js.

⚠ 2 — FeatureGate não é proteção real (risco de segurança)

Ocultar UI é conveniência. Todo endpoint do backend deve validar permissões independentemente.

⚠ 3 — Performance do FamilyForm (re-renderização)

Adotar React Hook Form para estado não-controlado e minimizar re-renderizações.

Regras de ouro para o engenheiro / IA implementador

✦ O frontend exibe, coleta e encaminha — nunca decide. Qualquer lógica de negócio pertence ao backend.

✦ Nunca hardcode um `programId` dentro de componentes. Use `useProgram()` + `FeatureGate`.

✦ Nunca duplique uma página por programa. Adicionar programa = nova config no backend.



✚ Validação client-side (formato) ≠ validação de negócio (elegibilidade).



✚ Permissões de navegação no frontend, operações no backend.

SIGAH – Arquitetura Frontend · Versão 3.0 · Responsabilidade: disposição de pastas e contratos de interface.
Regras de negócio: documento de arquitetura backend (etapa posterior)